NVT/ISA specifikacija školske 2026/27

1. Izmene specifikacije

U ovoj sekciji će stojati sve izmene specifikacije, u slučaju da smo nešto promašili. Povremeno pogledajte ovu sekciju za izmene, a svaka izmena će eksplicitno biti najavljena na Canvas platformi.

- 09.10.2026. - Inicijalna verzija specifikacije

2. Predmetno osoblje za vežbe

Asistenti:

- Nikola Velemir (velemir.sv8.2022@uns.ac.rs)
- Ivana Kovačević (kovacevic.ivana@uns.ac.rs)
- Milica Đumić (milicadjumic@uns.ac.rs)
- Luka Bursać (bursac.sv22.2022@uns.ac.rs)

3. OnlyKicks - skalabilno rešenje prodaja patika

Vaš zadatak je da napravite softversko rešenje koje pokriva ogromno tržište za prodaju patika. Softver mora da podrži mnogo korisnika, mnogo različitih patika i da radi performantno i u situacijama pod velikim opterećenjem.

Projekat razvijaju tri studenta u timu i nosi 55 bodova.

Na projektu postoje nivoi ocenjivanja svake funkcionalnosti: od najosnovnije implementacije, do napredne implementacije. Pošto je cilj ovog predmeta skalabilan sistem i rad pod opterećenjem, osnovne implementacije, to jest "samo da radi" će biti znatno slabije bodovane. Kolona sa minimalnim bodovima je vodič, ali se treba pročitati cela funkcionalnost. Kolone "bolje" i "odlično" ne otkrivaju kako se nešto radi, već samo na šta je potrebno obratiti pažnju.

3.1. Domenski zahtevi

3.1.1. Student 1

Prilikom pristupa platformi, neprijavljeni korisnik ima samo opciju registracije i login-a. Neprijavljeni korisnik popunjava formu za registraciju, u sklopu koje je u obavezi da zakači svoju profilnu sliku i navede podatke o platnoj kartici. Može postojati samo jedna platna kartica po korisniku i njen format studenti sami biraju, ali format mora biti konzistentan za svakog korisnika. Nakon uspešne registracije, korisnički nalog nije još uvek aktivan, već korisnik prima email na adresu koju je uneo pri registraciji, gde će mu biti dat link za aktivaciju naloga. Nakon što aktivira nalog, korisnik se može prijaviti.

Treba postojati i određena količina admin naloga koji ne mogu da se kreiraju, već će biti automatski popunjeni u bazi. Admin nalog ima pristup svim funkcionalnostima koje ima i običan korisnik, ali ima i dodatne funkcionalnosti koje su opisane dalje u tekstu. Potrebno je obezbediti izolaciju dozvola admin naloga i na serverskoj i na klijentskoj aplikaciji (npr. koristeći `JWT` tehnologiju). Potrebno je čuvati šifre u bazi na **bezbedan** i **efikasan** način (ovo nije bezbedonosni zahtev već zahtev opšte inženjerske kulture).

Admin nalog treba da ima i stranicu za pretragu korisnika. Neophodno je podržati mehanizme filtriranja po više parametara (filter se implementira na serverskoj, to jest glavnoj aplikaciji, a ne na prezentacionom sloju, to jest na web stranici). Parametri filtriranja mogu biti proizvoljni, ali ih je potrebno biti najmanje 3.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Registracija	• **Mora** se odraditi upload slike ▸ Slika se kači u istoj ACID transakciji • **Mora** uneti podatke o kartici • Mail se **mora** poslati • **Mora** sačuvati lozinku u bezbednom formatu	• Optimizovano slanje mejla tako da ne utiče na performanse i dostupnost glavne aplikacije • Upload slike bez obrade i čuvanja od strane glavne aplikacije	• Sprečena mogućnost siroče entiteta ako takvi entiteti postoje
Aktivacija i Prijava	• Aktivacija **mora** da funkcioniše • Prijava **mora** da funkcioniše	• Sprečeno trajno blokiranje korisnika da aktivira nalog ili da se ponovo registruje	...
Pretraga Korisnika	• **Mora** postojati neka vrsta paginacije • **Mora** postojati osnova za pretragu putem teksta • Pretraga **mora** da obuhvati i delimično poklapanje	• Vreme za pretragu ne sme značajno rasti sa rastom broja korisnika (sistem treba da podrži pretragu 1000000 korisnika)	• Optimizovana paginacija • Napredna pretraga na osnovu procene šta je najbolje za određenu kombinaciju parametara

3.1.2. Student 2

Admin može da upravlja fabrikama. Admin je u stanju da kreira, izmeni ili fizički obriše fabrike u sistemu. Prilikom kreiranja fabrike neophodno je popuniti formu sa: imenom, adresom, državom i gradom. Za pregled fabrika je neophodno implementirati efikasan mehanizam za pretragu. Neophodno je podržati mehanizme filtriranja po više parametara (filter se implementira na serverskoj, to jest glavnoj aplikaciji, a ne na prezentacionom sloju, to jest na web stranici). Parametri filtriranja mogu biti proizvoljni, ali ih je potrebno biti više.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Kreiranje	• **Mora** da bude implementirano	...	...
Izmena	• **Mora** da bude implementirano	• Ni jedna izmena ne sme biti poništena bez znanja admina	...

Brisanje	• **Mora** da bude implementirano kao fizičko brisanje	• Admin ne sme dobiti potvrdu uspešne akcije nad fabrikom koja ne postoji	...
Pretraga Fabrika	• **Mora** postojati neka vrsta paginacije • **Mora** postojati osnova za pretragu putem teksta • **Mora** sadržati neke filtere	• Vreme za pretragu ne sme značajno rasti sa rastom broja fabrika	• Optimizovana paginacija • Napredna pretraga na osnovu procene šta je najbolje za određenu kombinaciju parametara

3.1.3. Student 3

Kupac je u mogućnosti da prolazi kroz sve dostupne patike. Kupac je u mogućnosti da pretražuje patike koje ga interesuju. Pri prezentaciji svake pojedinačne patike, neophodno je prikazati sliku/e, proizvođača, naziv artikla i sve dostupne veličine. Neophodno je podržati mehanizme filtriranja po više parametara (filter se implementira na serverskoj, to jest glavnoj aplikaciji, a ne na prezentacionom sloju, to jest na web stranici). Parametri filtriranja mogu biti proizvoljni, ali ih je potrebno biti više. Svaki korisnik je u mogućnosti da ode na stranicu detalja o specifičnoj patici (gde se prikazuju slike u većoj rezoluciji, mogućnost kupovine, itd.).

Admin može da upravlja patikama. Može da kreira, izmeni i fizički obriše patike u sistemu. Prilikom kreiranja patike kao artikla, admin navodi naziv, bira proizvođača, dostupne veličine, inicijalno stanje za svaku veličinu i dve ili više slike patike. Funkcionalnost upravljanja proizvođačima nije potrebna, u redu je napraviti predefinisane proizvođače u okviru popunjavanja baze.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Pretraga patika i glavna stranica svih patika	• **Mora** postojati neka vrsta paginacije • **Mora** postojati osnova za pretragu putem teksta • **Mora** sadržati neke filtere • **Mora** ispoštovati zahtev o prikazu veličine patike na kartici patike (top 5 veličina, ne mora sve)	• Vreme za pretragu ne sme značajno rasti sa rastom broja patika	• Optimizovana paginacija • Napredna pretraga na osnovu procene šta je najbolje za određenu kombinaciju parametara
Detalji patika	• **Mora** da prikaže sve detalje patike,	...	...

	uključujući dobar način prikaza svih slika (carousel)		
Kreiranje patike	• **Mora** da bude implementirano ▸ Slike se kače u istoj ACID transakciji	• Upload slika bez obrade i čuvanja od strane glavne aplikacije	• Sprečena mogućnost siroče entiteta ako takvi entiteti postoje
Izmena patike	• **Mora** da bude implementirano	• Ni jedna izmena ne sme biti poništena bez znanja admina	• Asinhrono održavanje slika nakon izmene
Brisanje patike	• **Mora** da bude implementirano kao fizičko brisanje • Prilikom brisanja patike, brišu se i povezane slike	• Admin ne sme dobiti potvrdu uspešne akcije nad patikom koja ne postoji	• Asinhrono održavanje slika nakon brisanja

3.2. Razmena poruka i asinhrona komunikacija

3.2.1. Student 1

Kupac može da naruči proizvoljan broj artikala od jednom. Na stranici individualne patike, kupac može da izabere veličinu i potom da doda u korpu. Na stranici korpe, da bi se kupovina potvrdila, kupac treba da navede adresu gde njegova porudžbina treba da bude dostavljena.

Prilikom potvrde kupovine u korpi, potrebno je obezbediti i detekciju sumnjivih transakcija. Transakcija je sumnjiva ako dolazi sa nedozvoljenog broja kartice, ako je kartica blokirana, ako količina para pojedinačne transakcije prekoračava limit jedne transakcije za tu karticu, … (možete dodati proizvoljni broj uslova koji imaju smisla i ne morate nužno iskoristiti ove uslove, ali mora postojati barem 3). Program koji proverava da li je transakcija sumnjiva mora biti odvojen od glavne aplikacije i može biti napisan u proizvoljnom jeziku.

Kupovina i detekcija sumnjivih transakcija se **mora** vršiti asinhrono, preko `Kafka` (*log-based* broker) sistema. U slučaju da je detektovana sumnja, odbiti kupovinu i korisniku poslati izveštaj o tome na njegov mail. U slučaju da je sve OK, korisniku poslati potvrdu na mail o uspešnoj kupovini i obezbediti **bezbedan** mehanizam smanjenja količine svakog poručenog artikla u glavnoj bazi sistema (paziti na out-of-stock).

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Kupovina	• Koristi se `Kafka` • Postoji eksterni program za detekciju sumnjivih transakcija	• Istražiti *Outbox* obrazac i *CDC* (*change data capture*) tehnologiju	• Šta ako dva kupca kupe artikal u isto vreme, a nakon obrade prve

	• Kupovina se uspešno izvršava • Mail se uspešno šalje (zahteva odrađen posao studenta 3)	kontinualnog nadgledanja zapisa	kupovine, taj artikal više nije dostupan? • Ako je kupovina odbijena, šta raditi sa količinama? Šta ako je bilo više artikala u korpi?

3.2.2. Student 2

Patike ne nastaju same od sebe, već se proizvode u fabrikama. Potrebno je obezbediti "simulaciju" proizvodnje patika u fabrici, preko odvojenog programa. Komunikacija o proizvedenim patikama se **mora** vršiti asinhrono, preko `Kafka` (*log-based* broker) sistema. Izbor da li će fabrika slati jednu veliku poruku sa svim proizvedenim proizvodima ili pojedinačne poruke za svaki proizvedeni proizvod je ostavljena studentu. Potrebno je držati agregirani rezultat o proizvedenim patikama u glavnoj relacionoj bazi sistema. Svaka fabrika može da proizvodi bilo koji tip / marku / veličinu patike.

Pored agregiranog zapisa, potrebno je pratiti i podatke u stilu vremenske serije. Koristiti `InfluxDB` (preporučena verzija 2) kao bazu optimizovanu za čuvanje vremenski-orijentisanih podataka (eng. *time-series database*). Admin nalog mora imati pristup kontrolnoj tabli na kojoj se prikazuju `Grafana` grafici za broj proizvedenih patika u poslednjih: 1 dan, 3 dana, nedelju dana, mesec dana, 6 meseci i godinu dana. Omogućiti isti filter za specifičnu patiku.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Proizvodnja patika i analitika proizvodnje	• Definisan simulatorski program • Koristi se `Kafka` • Postoji `InfluxDB` instanca (ili bilo koja *time-series* baza) u kojoj se zapisuju podaci • Postoji admin dashboard realizovan preko `Grafana` alata	• Da li podaci uvek moraju biti prikazani na najmanje granularnom nivou? • Da li je pretraga patika odrađena na efikasan način?	• Istražiti *high-level* i *low-level* načine za prebacivanje podataka iz `Kafka` *topic*-a u bazu podataka. Da li se upisuje u relacionu bazu i `InfluxDB` na isti način? • Da li je potrebno čuvati neko stanje u međukoraku zapisa u relacionu bazu podataka? Koji je *low-level*, a koji *high-level* način da se ovo uradi?

3.2.3. Student 3

Svi mail-ovi koji se šalju za sve potrebne zahteva, se **moraju** slati asinhrono. Napraviti sveobuhvatni interni API, kao odvojen program, koji može da primi poruku i da je prosledi na broker poruka. Napraviti program odvojen od glavnog programa koji čita poruke sa brokera i šalje na mail (konzumer).

Omogućiti pokretanje proizvoljnog broja mail konzumera tako da ne remete jedan drugog. Time se postiže *load balancing* i horizontalna skalabilnost slanja mail-ova.

Napomena: u redu je slati sve mail-ove na isti mail iako pojedinačni registrovani korisnici u aplikaciji imaju različite mail-ove. Za ovo pronaći neki *development* alat.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
***API* za slanje mail-ova**	• Koristi broker poruka sa `AMQP` protokolom • Postoji program koji predstavlja *API* za slanje mail-ova • Postoji konzumer koji šalje mail-ove	• Šta ako konzumerski program padne? • Šta ako API program padne? • Šta ako je opterećenje sa bilo koje strane preveliko?	• Kako rešiti problem neuspešno obrađenih poruka?

3.3. Specijalni zahtevi

3.3.1. Student 2

Platforma OnlyKicks nudi mogućnost ostvarivanja šanse za kupovinom patika ograničene proizvodnje i tu ponudu naziva *SneakerDrop™*. Svaki kupac može da se prijavi na *SneakerDrop™* i ima šansu da bude izabran za dobitnika. Kupac se prijavljuje na artikal iz *SneakerDrop™* ponude tako što unosi svoju veličinu patike. U cilju zabrane eksploatacije ove pogodnosti, svaki klijent sme da se prijavi na maksimalno 2 artikla iz jedne *SneakerDrop™* ponude (koja može nuditi minimum 5, a maksimum 10 artikala). Sme postojati maksimalno jedna *SneakerDrop™* ponuda u bilo kom trenutku, a može da ne postoji ni jedna.

Prilikom objave *SneakerDrop™* ponude, admin specificira ime patike i brend patike (patike u *SneakerDrop™* ponudi su nevezane za obične patike jer se proizvode posebno za svakog pobednika; ne treba uvezivati funkcionalnost upravljanja patikama na ovoj stranici jer patike iz *SneakerDrop™*-a ne postoje van njega). Admin treba da postavi i propratni video (za potrebe projekta, video ne mora biti vezan za patike) koji služi kao reklamni materijal. Prilikom otvaranja stranice *SneakerDrop™* ponude i specifične patike, kupac može da pusti taj video. Završavanje *SneakerDrop™* ponude radi admin, manuelno. Kada se ponuda završi, dobitnici se za svaki artikal izvlače nasumično i automatski. Po jednom artiklu može postojati samo jedan dobitnik. Dobitnicima se šalje mail o pobedi.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Kreiranje Dropa	• **Mora** uneti zahtevane podatke • **Mora** uploadovati propratni video • Video se kači u istoj ACID transakciji	• Upload videa tako da ne opterećuje glavnu aplikaciju	• Obrađen slučaj kada se admin predomisli i poništi ili ne dovrši kreiranje ponude • Sprečena mogućnost siroče entiteta ako takvi entiteti postoje

Zaključivanje Dropa	• **Mora** biti implementirano • Mejl se šalje sinhrono	• Optimizovano slanje mejla tako da ne utiče na performanse i dostupnost glavne aplikacije • Dobitnici SneakerDropa se izvlače i obaveštavaju tačno jednom	…
Prijava na Drop	• **Mora** biti implementirana	…	• Rešiti slučaj velikog broja istovremenih prijava (voditi računa da svaki korisnik ima pravo na max 2 artikla)

3.3.2. Student 3

Svaka patika na stranici detalja može imati komentare koji kupci postavljaju. Svaki komentar u najmanju ruku prati slika i ime / inicijali / korisničko ime kupca. Neophodno je dozvoliti podkomentare (eng. *threaded replies*) do 5. nivoa. Komentari se mogu izmeniti i brisati. Ako je komentar izmenjen, neophodno je da piše momenat kada je izmenjen. Prikazati prvo najnovije komentare. Ako je admin komentarisao, neka postoji indikacija za to. Admin može da briše bilo koji komentar.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Upravljanje komentarima	• Može da kreira komentar i podkomentare • Koristi ih kao rekurzivnu strukturu (na serverskoj aplikaciji) • Može da izmeni komentar • Piše vreme promene ako je komentar izmenjen • Kaskadno briše komentar i sve ispod njega • Admin indikacija	• Kakve su privilegije korisnika i administratora pri radu sa komentarima?	• Šta se dešava sa potkomentarima prilikom brisanja natkomentara komentara? • Na koji način se rekonstruiše hijerarhija komentara i gde se pritom javlja opterećenje?

3.4. Reverse proxy

Ovo je posao za sve članove tima i potrebno je sinhronizovati se.

Neophodno je kompletnu infrastrukturu sistema (serverske komponente, kanale komunikacije, pomoćne aplikacije, itd.) efektivno "sakriti" iza *reverse proxy* sistema, kao i koristiti *reverse proxy* radi implementacije mehanizma balansiranja opterećenja. Balansiranje opterećenja implicira postojanje dve nezavisne glavne (serverske) aplikacije koje trče na različitim portovima, ali **ne** i dve replike glavne baze. Za tehnologiju koristiti `Nginx`. Serviranje statičkog sadržaja (što uključuje **izgrađeni** korisnički interfjes) isto obaviti isključivo preko `Nginx`-a.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Reverse proxy	• Mora postojati *reverse proxy* koji delegira pozive serverskoj aplikaciji • Ne sme se direktno gađati serverska aplikacija/e • Mora postojati barem **dve** serverske replike • Frontend mora biti serviran putem *reverse proxy*-ja	• Da li pristup *BLOB storage* linkovima ide kroz `Nginx`?	• Da li fabrike šalju informacije o proizvedenim patikama na isti server ili komuniciraju direktno sa brokerima?

3.5. Popunjavanje baze

Ovo je posao za sve članove tima i potrebno je sinhronizovati se.

Skalabilni sistemi rade sa jako mnogo podataka. Potrebno je obezbediti adekvatan skup podataka koji će dokazati da vaša implementacija OnlyKicks platforme radi kako treba.

U glavnoj bazi podataka, u trenutku KT-a, odbrane i testiranja performansi, mora da postoji minimum:
- 1.000.000 kupaca i 10.000 admina
- 500.000 artikala patika, gde svaka patika ima 5 random veličina u rasponu od 33 do 47. Količina za svaku veličinu da bude bar 5.
- 50.000 fabrika
- 5 komentara na svaki artikal patike, gde bar 1 artikal patika ima *threaded replies* dubine 5 i jedan artikal mora da ima bar 1000 komentara (zarad demonstracije)

U `InfluxDB` bazi podataka, u trenutku testiranja opterećenja i u trenutku odbrane mora da postoji minimum:
- 1.000 fabrika koje imaju podatke o proizvodnji, u poslednjih 2 godine, gde jedna fabrika proizvodi 50 različitih proizvoda, dva puta dnevno.
  - $1000 \text{ fabrika} * 2 \text{ puta dnevno} * 365 \text{ dana} * 2 \text{ godine} * 50 \text{ proizvoda} = 73.000.000 \text{ zapisa}$

Napomena: nije potrebno imati sliku za svaki entitet koji je ubačen sintetički da bi se ispunio ovaj zahtev, ali za određenu količinu jeste potrebno da bi se demonstrirala funkcionalnost.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Popunjavanje baze	• Baza koja nema podatke ograničava studente na dobijanje bodova samo na neskalabilne funkcionalnosti	• Da li baze podataka učitavaju podakte brže ako ih ubacujemo u paketima ili jedan po jedan?	• Da li podatke generisati *kompletno* nasumično ili voditi računa o sinhronizaciji između primarnih i stranih ključeva? • Da li je ubacivanje gigabajta podataka u bazu moguće od jednom?

3.6. Keširanje

U okviru razvoja OnlyKicks platforme, potrebno je uočiti slučajeve korišćenja kojima se često barata i primeniti keširanje na njih. Kvalitetno primenjeno keširanje omogućava znatno ubrzanje sistema.

Savet: dobro urađeno testiranje performansi može otkriti mesta gde se keširanje može vršiti.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Keširanje podataka	• Primenjen keš i objašnjeno gde je primenjen	• Koje operacije i delovi sistema imaju najviše koristi od keširanja i zašto?	• Kada keširani podaci prestaju da budu validni i kako se obezbeđuje njihovo ažuriranje?

3.7. Testiranje performansi sistema

3.7.1. Testiranje glavne aplikacije

Putem alata `Locust`, testirati performanse glavne aplikacije sistema. Testiranje se radi nad konkretnim slučajevima korišćenja, koji se mogu sastojati iz jednog ili više ulančanih *endpoint*-a. Primer: kreiranje patike (verovatno jedan *endpoint*); primer: uspešan login novog korisnika (korisnik se prvo registruje, potvrdi mail i onda login-uje što je 3 ulančana *endpoint*-a). Studenti sami određuju koji su najbitniji delovi aplikacije koje je potrebno testirati, ali mora najmanje 3 *endpoint*-a biti pokriveno (bili oni zasebni ili deo ulančanog slučaja korišćenja).

Testiranje treba otpočeti od malog broja korisnika i postepeno inkrementirati broj istovremenih korisnika. Neophodno je definisati kriterijum obustavljanja testiranja, odnosno do koje latencije ili procenta grešaka se testiranje aplikacije vrši. Primer 1: *endpoint* za registraciju korisnika može da podrži obradu 5000 korisnika, gde je *p99* jedna sekunda i 1% zahteva završi sa *time-out* statusom (postigli smo ok performanse, gurati ovaj *endpoint* dalje). Primer 2: *endpoint* za kreiranje patike može da podrži 8000 korisnika, ali na 8500 korisnika *p99* je 20 sekundi, a stopa *time-out* statusa je 40% (8000 je realan limit i ovo dokumentovati). Ne inkrementirati fiksno, već raditi binarnu pretragu.

Nakon dokumentovanja svojih zapažanja, traži se da studenti razmotre pravce poboljšanja svoje aplikacije, i ako uoče znatne degradirane performanse, ulože napor u unapređenje aplikacije i ponovo testiraju performanse.

Sva zapažanja neophodno je sabrati u izveštaj u slobodnoj formi. Izveštaj treba da sadrži tekst opisa toga što se testira, grafikone i listinge koji mogu doprineti prikazivanju celokupnog procesa testiranja. Izveštaj treba da sadrži specifikaciju računara (ili više njih) na kom je pokrenuto rešenje i gde su pokretani testovi. Izveštaj mora biti *Markdown* formata (pregledno na GitHub platformi). Izveštaji ostalih formata se neće razmatrati. Mora postojati tačno **jedan** izveštaj gde će stojati zapažanja od svih studenata, smisleno grupisano.

Savet: Serversku aplikaciju, sa svojim komponentama pokrenite na jednom računaru, a na drugom računaru pokrenite `Locust`. Neophodno je da se računari nalaze u istoj mreži kako bi međusobna komunikacija bila brza.

Napomena: ako smatrate da je neki alat adekvatniji od `Locust`-a, slobodno ga koristite, uz minimalno objašnjenje zašto.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Testiranje glavne aplikacije	• Izveštaj mora postojati • Podele tipa 10/100/1000/10000 ne sme da se rade. Neophodno je da se slučaj korišćenja / endpoint gura postepeno	• Da li konfiguracioni parametri relacione baze ili `InfluxDB` baze utiču na performanse? • Da li `Nginx` parametri utiču na latenciju ili stepen grešaka?	• Da li konfiguracioni parametri sistema ili `Docker` okruženja utiču na performanse?

3.7.2. Testiranje asinhrone komunikacije

Potrebno je izvršiti i testiranje asinhrone komunikacije:

- testirati koliko poruka je neophodno da bi se red poruka “zapušio”.
- testirati da li horizontalnim dodavanjem eksternih programa povećavaju propusnu moć bez rušenja korektnog funkcionisanja aplikacije.

Implementaciona očekivanja

Zahtev	Minimum	Bolje	Odlično
Testiranje asinhrone komunikacije	• Mora se testirati koliko je neophodno da se asihnhroni komunikacioni kanal “zapuši”	…	• Izvršeno je testiranje komunikacionih kanala prilikom dodavanja konzumer-a za brokere poruka (student 3) • Teoretski je objašnjeno kako se

			`Kafka` sistemi skaliraju i kako `Kafka` sistemi mogu da podrže više poruka

3.8. Kontrola verzija

Neophodno je koristiti `Git` sistem za kontrolu verzija, a udaljeni repozitorijum mora biti `GitHub`. Neophodno je u `README.md` dokumentu napisati kako pokrenuti sistem, kako popuniti sistem testnim podacima (to jest kako popuniti baze), koje su tehnologije korišćene i informacije o studentima. Neophodno je da se finalna verzija projekta tokom kontrolne tačke i odbrane nalazi na `main`, `master` ili `develop` grani. Prethodne korake je neophodno ispratiti jer u suprotnom rešenje neće biti pregledano pre odbrane, te tim gubi mogućnost odbrane pred predmetnim asistentom u datom roku.

3.9. Kontrolna tačka

Kontrolna tačka se brani u terminu vežbi, ili u slučaju prevelike zainteresovanosti, u nekom dodatnom terminu.

Kontrolna tačka nosi 20 bodova i bodovi sa kontrolne tačke se ne mogu vratiti na finalnoj odbrani. Postojaće i konsultativni termin pre kontrolne tačke koji se neće ocenjivati, gde će studenti moći da dobiju povratne informacije u vezi svog projekta. Datumi će biti naknadno objavljeni.

Šta je neophodno uraditi do kontrolne tačke:
- Student 1 i stari studenti: **3.1.1.**
- Student 2: **3.1.2.** , **3.3.1.**
- Student 3: **3.1.3.** , **3.3.2.** + demonstracija da je *threaded replies* moguć
- Zajedničko: **3.5.** (bez `InfluxDB`)

Ako student 3 stigne da napravi i API za slanje mailova, mail-ovi se mogu slati u okviru traženih funkcionalnosti. Ako ne, nije potrebno slati mail-ove, ali će ova funkcionalnost morati da se demonstrira na konačnoj odbrani.

Pre kontrolne tačke će postojati konsultativni termin vezan specifično za nju, gde će studenti moći da prezentuju neke delove svog rešenja i vide da li su uradili dobar posao za samu kontrolnu tačku.

3.10. Bonus

U ovoj tabeli stoje bonus zahtevi koji studenti mogu da odrade za dodatne bodove. Bonusi **nisu** obavezni. Svaki bonus se mora naznačiti da je urađen prilikom odbrane i potrebno je objasniti njegovu implementaciju. Svaki bonus nosi 10 bodova ako je dobro odrađen (ako nije, nosi 0) i može da se prelije u ispitne bodove, time olakšavajući ispit.

Student 1	Student 2	Student 3
Implementirati *rate-limiting* algoritam za celu serversku aplikaciju, po korisniku.	U okviru *SneakerDrop™* ponude, podržati videe veće od 2GB na skalabilan način.	Implementirati skalabilni sistem za ocenjivanje komentara (lajkovi).

3.11. Opšte projektne informacije

- Svaki kupac u aplikaciji barata sa beskonačno mnogo novca.
- Sve tehnologije su proizvoljne sem tamo gde je naznačeno.
- Preporučeno je praviti dijagram sistema **što pre**, zbog ranijih povratnih informacija.
- Neka vrsta skripte koja pokreće sve različite komponente sistema je poželjna. Ko želi može primeniti `Docker` i `Docker Compose` u ovu svrhu (i time dosta olakšati život sebi), ali njihovo korišćenje neće biti zahtevano.
- **Bilo koji zahtev** koji traži *BLOB* datoteke, **ne sme** biti realizovan kroz fajl sistem, niti kroz glavnu (serversku) aplikaciju, već se mora primeniti adekvatna tehnologija.
- Dodati `isa-asistent-2026` kao člana na GitHub repozitorijumu i držati repozitorijum na privatnoj vidljivosti. Ne dodavanje `isa-asistent-2026` GitHub naloga kao člana repozitorijuma ili držanje repozitorijuma javnim vas diskvalifikuje od odbrane u datom roku.
- Pre početka izrade projekta, biće potrebno upisati članove timova u tabelu. Timovi su **tročlani**, pa studenti koji upišu dva člana će biti preraspoređeni u timove od troje. U slučaju da ostane dvočlanih timova, za njih se student 2 izbacuje i njegove funkcionalnosti nije potrebno implementirati. Pošto ne postoji proizvodnja patika, postaviti ukupno 100.000 artikala patika, gde svaka patika ima 5 random veličina u rasponu od 33 do 47. Količina za svaku veličinu da bude bar 5.
- Za stare studente, funkcionalnosti koje je potrebno odraditi su:
  - domenski zahtev sva tri studenta (3.1.1. , 3.1.2. , 3.1.3.)
  - asinhron zahtev studenta 2 (3.2.2.)
  - specijalni zahtev studenta 3 (3.3.2.)
  - sve zahteve koji nisu specifični za jednog studenta

3.12. Generalna raspodela po studentima

Studenti sami biraju koji će biti student na projektu. Tabela o generalnim raspodelama ne predstavlja količinu posla, već koje delove sistema određeni student obrađuje. Iako treći student ne radi sa `Kafka` platformom, radi sa nekim brokerom poruka. Celo gradivo predmeta je pokriveno kod svakog studenta, ali na različite načine.

Funkcionalnost / tehnologija	**Student 1**	**Student 2**	**Student 3**
CRUD / domenski zahtev	korisnici	fabrike	patike
Specijalni zahtev		*SneakerDrop™*	komentari
Event stream platforma (`Kafka`)	☑	☑	
Broker poruka			☑
`InfluxDB` + `Grafana` Dashboard		☑	
BLOB storage	☑	☑	☑
Pretraga	☑	☑	☑
Pisanje eksternih programa	☑	☑	☑

Keširanje	☑	☑	☑
`Nginx`	☑	☑	☑
Popunjavanje baze	☑	☑	☑
Testiranje performansi	☑	☑	☑

4. Odbrana

Na odbrani će biti potrebno pokrenuti sve delove aplikacije i pripremiti izveštaj. Ovo uključuje i asinhrone slojeve. Više detalja će biti objavljeno kada se odbrana približi.

Pre same odbrane, svaki student će raditi eliminacioni zadatak. Uslov za odbranu projekta u datom roku je uspešno odrađen eliminacioni zadatak. Ako student padne na eliminacionom, može da pokuša odbranu u sledećem roku. Eliminacioni zadatak će biti realizovan kao test na papiru.

Postojaće tri roka odbrane: kraj januara / početak februara, maj i septembar. U maju se bodovi **skaliraju** na 90%, a u septembru na 80%.

5. Akademska čestitost

Akademska čestitost podrazumeva samostalno pisanje radova (teksta, programskog koda i slično) uz striktno poštovanje tuđih autorskih prava. Ovaj pojam i obaveza su sastavni deo Zakona o visokom obrazovanju.

U okviru ovog predmeta to podrazumeva da studenti samostalno pišu sopstveni rad. Pomoć drugih studenata u obliku direktno preuzetih delova teksta ili programa nije dozvoljena. Kako se ovaj projekat radi u timu, odgovornost za nepridržavanje principa akademske čestitosti snose svi članovi tima.

AI alati su dozvoljeni, ali ako student bude uhvaćen da ne zna šta mu je AI alat napisao, snosiće posledice zavisno od kompleksnosti funkcionalnosti.